This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Attention-aware Inference Optimizations for Large Vision-Language Models with Memory-efficient Decoding

Fatih Ilhan[1], Gaowen Liu[2], Ramana Rao Kompella[2],
Selim Furkan Tekin[1] Tiansheng Huang[1], Zachary Yahn[1], Yichang Xu[1], Ling Liu[1]
[1]Georgia Institute of Technology, [2]Cisco Research
filhan@gatech.edu, {gaoliu,rkompell}@cisco.com,
{stekin6,thuang,zachary.yahn,xuyichang}@gatech.edu, ling.liu@cc.gatech.edu

Abstract

Large Vision-Language Models (VLMs) have achieved remarkable success in multi-modal reasoning, but their inference time efficiency remains a significant challenge due to the memory overhead during decoding, especially when the query and answer of VLMs consist of long sequences of visual and text tokens. This paper presents AttentionPack, an adaptive and attention-aware optimization framework tailored for large vision-language models with improving memory-efficiency during decoding, focusing on addressing the challenges due to the increased high number of visual inputs and interactions, particularly in long-context tasks with multiple high-resolution images or videos. AttentionPack is novel in two aspects: (i) We introduce a multi-head attention compaction method for economically storing key and value matrices by exploiting the implicit low-rank structure, and (ii) we develop a token-specific attention-aware decompression mechanism to reduce latency overhead. Experimental results on multiple benchmarks demonstrate that AttentionPack improves memory efficiency by up to 8x, enabling higher batch sizes and faster batch inference while preserving the model output quality or longer context lengths for superior retrieval performance. We also report the effectiveness of AttentionPack combined with eviction, quantization and kernel fusion, showing further efficiency gains for resource-limited environments.

1. Introduction

Large Vision-Language Models (VLMs) with billions of parameters trained on vast amounts of multi-modal data have achieved remarkable success across both computer vision and natural language processing domains [20, 26]. These models are reshaping how we interact with AI systems in ways that cannot be achieved solely by language or vision uni-modal models. Currently, a majority of popular large VLMs rely on sequential visual representation, where images are transformed into hundreds or thousands of tokens when feeding them to LLMs along with language prompts [18, 20].

However, large VLM inference can be prohibitively expensive, especially for tasks requiring long context, such as in dialogue systems, document understanding and video analysis. A significant amount of temporary intermediate data (containing the key and value vectors of past tokens for each batch instance, layer and attention head) is stored to prevent recomputation during decoding, known as KV cache, with a size increasing with the sequence length, number of dimensions and batch size. Although this technique reduces computation during a single decoding step, one needs to fetch ever-growing data, only to perform vector-to-matrix operations. This may result in more time being spent loading these vectors to GPU memory rather than for the computation itself, especially for long-context tasks, causing underutilization of the compute and high inference latency. For instance, an large VLM with 13B parameters [20], processing 16 images each with 256 tokens and a batch size of 64, may require around 214 GB for inference at half precision. To this end, in this work, we tackle the challenges of efficient storage, compression and retrieval of these intermediate structured data under stringent memory constraints.

Recent studies aiming to directly reduce the size of this cache can be categorized into two directions: (i) token eviction along the sequential axis, and (ii) compression through quantization to represent information with fewer bits. Eviction techniques [21, 32] focus on developing algorithms to release tokens from the cache based on certain heuristic criteria such as attention scores, norm of key vectors, or token categories. However, the memory footprint reduction remains limited as the number of stored dimensions remains the same. Quantization-based approaches [13, 15] represent the vectors with lower bit precision but struggles with outlier values and hardware compatibility. large VLMs leverage the advanced emergent capabilities inherent in their language components, however, only a few recent works focus on analyzing how

the behavior of visual tokens differ and capitalizing these differences to enhance efficiency [5]. To this end, we focus on thoroughly understanding this aspect, specifically, analyzing the low-rank structure of visual/textual tokens and significantly reducing the memory footprint by compressing them along the hidden dimension axis without evicting tokens.

AttentionPack capitalizes on the fact that stored key and value vectors, particularly for visual tokens, tend to have low intrinsic rank, which can be utilized to represent the data with fewer dimensions using singular value decomposition (SVD). Based on our analysis, we conclude that these vectors can be compressed up to 8 times for large VLMs such as LLaVA variants and QwenVL without hurting model output quality. In addition, we develop an attention-aware decompression technique to reduce the overhead due to decompression at decoding stage. Thanks to the reduced cache size, users can operate with larger batch sizes to improve throughput or increase context length to enhance retrieval. We perform experiments on various image and video tasks with LLaVA1.5 [20], QwenVL [2] and VideoLLaVA [18] variants, which demonstrates the effectiveness of our approach. We further analyze the behavior of compression and decompression, their sensitivity to different parameters, latency breakdown and compatibility with other techniques.

2. Related Work

A wide range of prior research efforts have focused on either architectural modifications or fine-tuning adjustments [1, 16] for efficiency. For instance, approaches such as Reformer [16], Longformer [3], and Performer [7] aim to improve the scalability of attention mechanisms when dealing with long sequences by token grouping and approximating attention patterns. In the vision domain, A-ViT [29] proposes a learnable token dropping mechanism for vision transformers during inference. While these techniques offer reductions in the computational cost of attention operations, they do not address the memory bottlenecks that arise from storing the key-value cache. Notably, grouped-query attention (GQA)[1] reduces the cache footprint and accelerates inference speed by sharing the same key and value matrices across heads. Other efforts have explored improving memory management and I/O efficiency to reduce latency. Low-level optimizations such as FlashAttention [8] and PageAttention [17] minimizes costly memory transfers during attention computation or optimizes resource allocation using paging techniques. Nevertheless, despite these advancements, the size of the cache remains same, continuing to pose a major source of memory bottleneck for long-context inference.

In addition to these strategies, model compression techniques such as pruning [14, 22] and low-rank approximations [30] have been explored as indirect methods to reduce the size of the key-value cache by shrinking the hidden dimension size of the model's weights. However, these methods generally require extensive pre-deployment optimization on a representative dataset to preserve model performance, posing limitations for users who seek to run inference without engaging in additional model fine-tuning. To further alleviate cache memory constraints, token merging [19] or quantization-based techniques have also been proposed [13, 15]. For example, KVQuant [13] introduces a sensitivity-aware, non-uniform quantization strategy that operates after outlier values in the cache have been eliminated. Similarly, GEAR [15] leverages a SVD-based decomposition of residuals, enabling more accurate reconstruction of dequantized key-value vectors. While effective, these approaches often rely on hardware-specific low-level optimizations, and assumptions regarding the minimal impact of outliers or newly generated vectors on overall performance.

In recent years, research efforts has focused toward token eviction policies designed to manage the memory for key-value cache more effectively by selectively discarding certain tokens. H2O [32], for example, implements a sliding window approach that retains only the most recent tokens while discarding others based on accumulated attention scores. Scissorhands [21] follows a different strategy by identifying and removing tokens that consistently receive low attention scores during inference. In the specific case of LVLMs, several studies have investigated redundancy within visual tokens and proposed token dropping techniques tailored to this modality. FastV [5] introduces an eviction criterion based on attention scores computed in the final decoding step, while Look-M [27] applies eviction based on cumulative attention scores accumulated over time. Despite their promising results, these eviction-based methods face challenges in substantially reducing memory usage due to key-value cache without introducing unacceptable levels of information loss, as the hidden dimension size of cached vectors remains constant. To overcome this limitation, recent work has explored decomposition of key and value weight matrices to reduce cache size along the hidden dimensions [4].

3. Methodology

This section provides a comprehensive overview of our methodology. We begin with a formal description of the key-value caching process in LVLMs, followed by an analysis on the low-rank structure of stored vectors. Next, we detail the multi-head compression of these vectors using matrix decomposition. Lastly, we thoroughly explain the attention-aware decompression stage and provide an algorithmic description of the overall process during decoding. We provide a detailed algorithmic description in Appendix 6.

Consider an LVLM such as LLaVA [20] which accepts visual $\mathbf{X_v}$ and text inputs $\mathbf{X_t}$. A pretrained visual encoder can be used as backbone to extract features from visual inputs $\mathbf{Z_v} = g(\mathbf{X_v}) \in \mathbb{R} \in T_v \times D_v$, followed by a linear projection using matrix $\mathbf{W}_p \in \mathbb{R}^{D_v \times HD}$, $\mathbf{H_v} = \mathbf{W}_p\mathbf{X_v}$ to

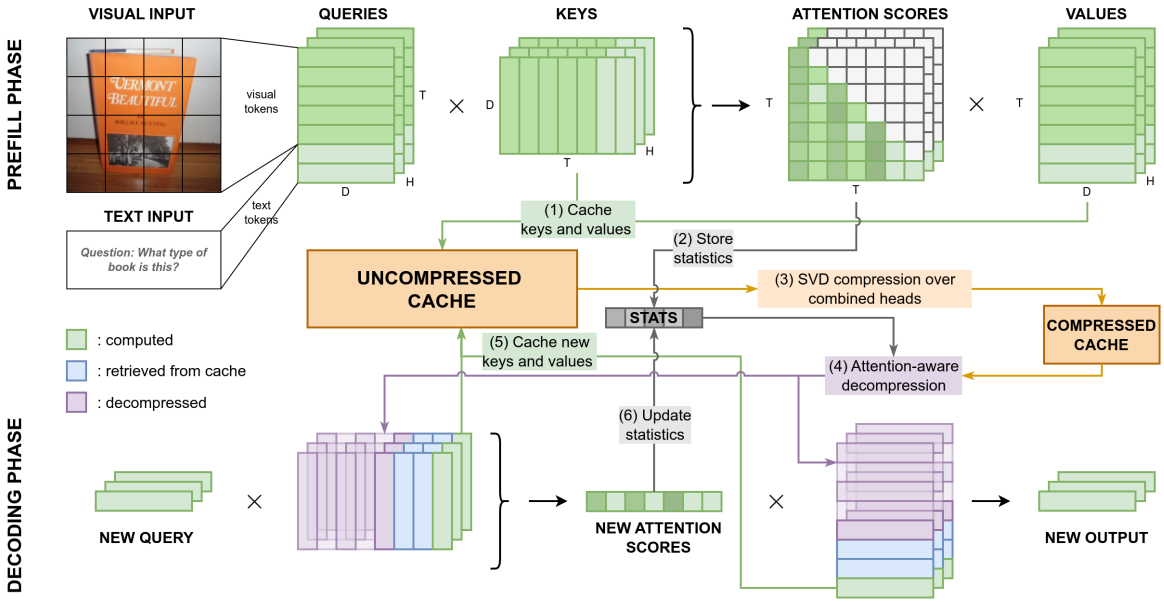


Figure 1. The schematic of the workflow during inference. After the prefill phase, at each decoding step, we first compress the cache along combined heads. We perform attention-aware partial decompression before attention score computation.

achieve the same dimensionality with embedded text input vectors $\mathbf{H_t} \in \mathbb{R}^{T_t \times HD}$. Here, T_t and T_v are the number of textual and visual tokens. In the attention block within a layer, weights $\boldsymbol{\theta} = \{\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o \in \mathbb{R}^{HD \times HD}\}$ are used for query, key, value, and output transformations. Here, H and D are the number of heads and hidden dimensions.

During decoding, attention computations require access to key and value vectors from past. These vectors are stored in a key-value cache, represented as $\mathcal{C} = \{\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times HD}\}$, where $T = T_v + T_t$ is the number of stored tokens after the prefill phase. At each decoding step, new key $\boldsymbol{k}$ and value $\boldsymbol{v}$ vectors are appended to the cache and used for attention calculations. While this technique reduces computational overhead, it potentially leads to memory constraints in long-context scenarios as storing these vectors in system memory introduces significant latency due to data transfers between host (CPU) and device (GPU) memory. To address this, our approach compresses these vectors, enabling efficient storage in GPU memory and eliminating offloading overhead. Figure 1 provides an overview of the workflow.

3.1. Multi-head Compression of Key/Value Vectors

Reducing the size of key-value cache allows for (i) processing queries in larger batches, leveraging parallelization to minimize overall processing time, and (ii) enabling long-context inference in GPU-constrained environments. To achieve this, we apply compression over the uncompressed key and value vectors to significantly shrink their sizes. Our analysis on the first layer of LLaVA1.5-7B [20] over OCR-VQA [23] samples ($T_v = 576$), as illustrated in Figure 2, demonstrates that the stored vectors exhibit an inherent low-rank structure, which can be exploited for dimensionality reduction using singular value decomposition (SVD). Additionally, we find that compression is more effective when low-rank decomposition is applied to the combined cache along multiple heads rather than treating each head independently. However, handling visual and textual tokens separately is critical, as they originate from different modalities and applying SVD indiscriminately across both results in suboptimal compression. To this end, we apply the procedures separately for visual/textual input tokens. To achieve higher compression rates, before applying low-rank SVD, we first merge the vectors along the head axis, allowing shared information across heads to be compressed together.

This process decomposes each stored matrix into two low-rank components: $\mathbf{K}_* \approx \overline{\mathbf{K}}_* \mathbf{D}_{k*}$ and $\mathbf{V}_* \approx \overline{\mathbf{V}}_* \mathbf{D}_{v*}$, where $\overline{\mathbf{K}}_* \in \mathbb{R}^{T_* \times R_{k*}}$ represents the compressed key cache, $\mathbf{D}_{k*} \in \mathbb{R}^{R_{k*} \times HD}$ serves as the key decompression matrix, $\overline{\mathbf{V}}_* \in \mathbb{R}^{T_* \times R_{v*}}$ is the compressed value cache, and $\mathbf{D}_{v*} \in \mathbb{R}^{R_{v*} \times HD}$ serves as the value decompression matrix. Each column of the first low-rank component is scaled using the computed singular values after SVD. Here to notate, $* = v$ for visual tokens and $* = t$ for textual tokens. The rank values R_{kv}, R_{kt} and R_{vv}, R_{vt} for key and value matrices are tunable parameters and allow adjustments based on the available memory budget. We illustrate the compression process in the top half of Figure 3 (Number of dimensions in the figure are for illustration purposes). For instance, the storage requirement for key vectors of visual tokens at a single layer and for one input decreases from $T_v HD$ to $T_v R_{kv} + R_{kv} HD$ after applying compression for $R_{kv} << \min(T_v, HD)$, resulting in the following compression ratio:

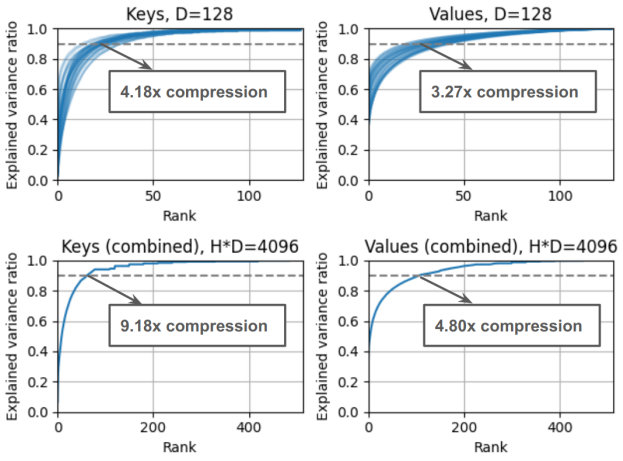


Figure 2. Rank vs explained variance ratio without/with combining along head axis before compression for key and value vectors.

$c_{kv} = \frac{T_v HD}{T_v R_{kv} + R_{kv} HD}$. As a numerical example, when $T_v = 1000$, $H = 40$, $D = 128$, and $R_{kv} = 64$, the resulting memory footprint after compression is around 13x lower.

3.2. Attention-aware Decompression

Before computing attention scores, the compressed vectors are decompressed by multiplying them with the corresponding decompression matrices. While compression is performed once after the prefill phase and periodically during decoding, decompression occurs at every step and introduces latency overhead. However, the memory savings from compression enable larger batches, allowing parallelization to mitigate this overhead and speed up overall computation. In single-instance inference, however, the added latency can increase by up to 30%, depending on the context length.

To mitigate this overhead, we introduce attention-aware decompression, leveraging that not all stored tokens contribute equally at every decoding step. While all tokens are initially compressed to the same low rank, fewer ranks can be used during decompression for tokens (image regions in visual input and words in textual input) that are less influential in the final output computation. To implement this, we track scaled accumulated attention scores using moving average with parameter $\alpha \in [0, 1)$ after computing attention scores at each step. At decoding step t, the importance score for each token at step t_p for $t_p \in \{1, ..., t\}$ is computed as:

$$\mathcal{I}_{t_p} \leftarrow \alpha^{T_q} \mathcal{I}_{t_p} + (1 - \alpha^{T_q}) \frac{\sum_{t'=t-T_q}^{t} A_{t' t_p}}{T_q}, \qquad (1)$$

where T_q is the number of new tokens, which is one except an external input with multiple tokens is provided by the user. Here, $A_{t' t_p}$ represents the attention weight from t_p in decoding step t', and averaged across heads. Setting $\alpha = 0$ only takes the most recent attention score into account.

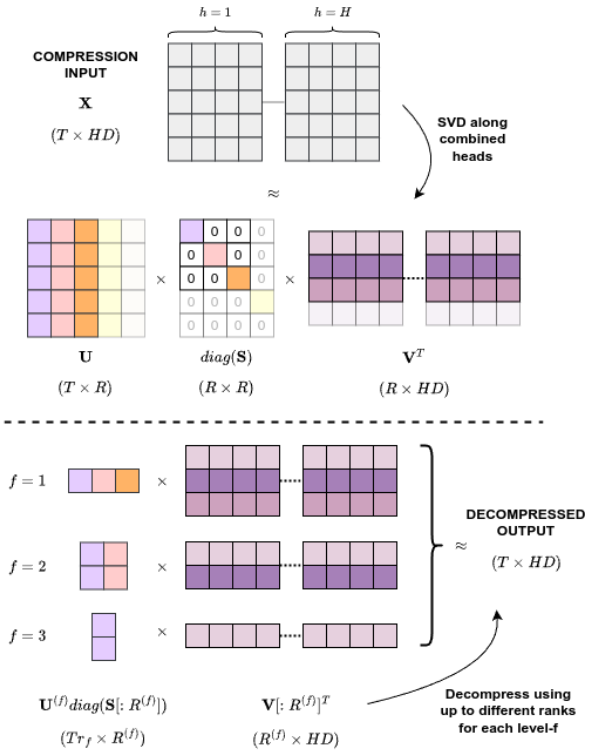


Figure 3. Visualization of compression and partial decompression.

Tokens with the highest scores are decompressed using the original compression rank, while those with lower scores are decompressed with reduced ranks. This strategy significantly decreases the decompression FLOPs. For instance, in the key cache for visual tokens, this reduces the FLOPs from $2T_v HDR_{kv}$ to $2T_v HD \sum_{f=1}^{F} r_f R_{kv}^{(f)}$, where F is the number of groups, each with $r_f T_v$ tokens and decompression rank of $R_{kv}^{(f)}$. We set the group $f = 1$ having the highest importance and decompress with $R_{kv}^{(1)} = R_{kv}$, i.e. the rank used in compression. As an example, for $F = 2$, $T = 1000$, $r_1 = 0.1$, $r_2 = 0.9$, $R_{kv}^{(1)} = 64$, $R_{kv}^{(2)} = 16$, where the 90% of tokens with the lowest scores are decompressed with the 25% of the rank, the FLOPs reduce by 67.5%.

4. Results

In this section, we present the results obtained across various datasets, along with comparisons to state-of-the-art efficiency techniques and an in-depth analysis of memory and computational costs. To summarize, our approach achieves around 80% reduction in cache size (5.1x smaller) on LLaVA1.5-7B and LLaVA1.5-13B, 63% reduction (2.8x smaller) on QwenVL-Chat-7B and 88% reduction (8.1x smaller) on VideoLLaVA through compression while maintaining performance. The reduced memory footprint enables up to 74% faster decoding in batch inference with attention-aware decompression for image and 60% for video QA.

Model	Method	Cache Memory Reduction (avg. per instance)	Average Throughput Change	Image QA Datasets A-OKVQA (acc.)	OCR-VQA (rouge-l)	MMMU (acc.)
LLaVA1.5-7B	Full KV Caching	–	–	76.64	51.05	34.68
	FastV [5] ($k = 5, e = 50\%$)	1×	+29%	76.72	**52.62**	34.27
	ScissorHands [21] ($e = 50\%$)	2×	+32%	75.50	50.08	31.66
	H2O [32] ($e = 50\%$)	2×	+32%	76.36	52.09	32.15
	Minicache [19]	4.51×	+44%	76.54	51.93	33.75
	AttentionPack ($R_{kv} = R_{vv} = 64$)	5.09×	+54%	**76.88**	52.44	**34.59**
	AttentionPack ($R_{kv} = R_{vv} = 32$)	**7.24×**	**+65%**	75.91	51.69	32.79
LLaVA1.5-13B	Full KV Caching	–	–	81.40	55.01	36.40
	FastV [5] ($k = 5, e = 50\%$)	1×	+20%	81.17	**53.50**	**36.61**
	ScissorHands [21] ($e = 50\%$)	2×	+24%	77.48	51.32	31.90
	H2O [32] ($e = 50\%$)	2×	+24%	79.10	52.49	33.01
	Minicache [19]	4.35×	+38%	81.20	53.18	35.52
	AttentionPack ($R_{kv} = R_{vv} = 64$)	5.17×	+43%	**81.25**	53.22	36.38
	AttentionPack ($R_{kv} = R_{vv} = 32$)	**7.30×**	**+50%**	80.00	51.92	34.40
QwenVL-Chat-7B	Full KV Caching	–	–	75.67	70.5	35.82
	FastV [5] ($k = 2, e = 50\%$)	1×	+45%	75.10	66.43	**35.80**
	ScissorHands [21] ($e = 50\%$)	2×	+51%	74.60	67.10	35.14
	H2O [32] ($e = 50\%$)	2×	+51%	74.98	67.15	35.05
	AttentionPack ($R_{kv} = R_{vv} = 64$)	2.77×	+61%	**75.33**	**68.45**	35.72
	AttentionPack ($R_{kv} = R_{vv} = 32$)	**4.02×**	**+74%**	75.17	68.02	35.67

Table 1. Image QA results on AOKV-QA, OCRKV-QA and MMMU with LLaVA1.5-7B, LLaVA1.5-13B and QwenVL-Chat-7B. We report evaluation metrics along with cache memory and throughput statistics for full KV caching, ScissorHands [21], H2O [32], FastV [5], Minicache [19] to compare with AttentionPack.

4.1. Experiment Setup

We experiment on five datasets from different image/video tasks A-OKVQA [25], commonsense and world knowledge task, (ii) OCR-VQA [23], optical character recognition task, (iii) MMMU [31] a multi-discipline multi-modal reasoning benchmark, (iv) MSVD-QA [28] and (v) MSRVTT-QA [28] containing short video clips with multiple question-answer pairs for each video. For image QA tasks, we experiment with 7B and 13B variants of the LLaVA1.5 [20] and QwenVL-Chat-7B [2]. We also consider Qwen3VL-8B-instruct for experiments with grouped query attention [11]. In video QA tasks, we consider VideoLLaVA-7B, which has a similar architecture with LLaVA. For comparisons, we consider: H2O [32], Scissorhands [21], FastV [5] and Minicache [19]. We perform attention-aware decompression over value cache with $F = 2, r_1 = 0.25, r_2 = 0.75, R_{vv} = R_{vv}^{(1)} = 4R_{vv}^{(1)}$. In initial experiments, $\alpha \in [0.05, 0.75]$ gave robust and stable results and we set $\alpha = 0.25$.

4.2. Comparisons

We report the main experiment results in Tables 1 and and 2 for image and video question-answering (QA) tasks. For QA tasks with text output we report ROUGE-*L* (F1) scores, where *L* is the length of the longest common subsequence between prediction and target. For multiple choice question tasks, we report accuracy. We also report average cache size per instance and decoding throughput change compared to baseline with batch inference. For QA pair instances corresponding to the same image/video, we perform inference together without releasing the key-value cache. For H2O and ScissorHands, we evict 50% of tokens following the algorithms on attention-score based eviction policies. In Minicache, we follow the settings applied for LLaMA variants from the paper [19], on the visual tokens. In FastV, we apply 50% token skipping starting after the second or fifth layer but as the tokens are not released and memory usage due to the key-value cache stays the same.

With LLaVA1.5-7B, AttentionPack performs +0.16%, -0.18% and +0.32% with respect to the closest comparison across three datasets while enabling 5.09 times smaller cache size on average per instance. For instance, the total (textual+visual) size for full KV caching per sample is around 328.2 MB, and is reduced to 64.5 MB with AttentionPack. Likewise, with LLaVA1.5-13B, AttentionPack performs +0.08%, -0.28% and -0.23% with respect to the closest comparison across three datasets while enabling 5.17 times smaller cache size. Lastly, with QwenVL-Chat-7B, AttentionPack performs +0.35%, +1.30% and -0.08% across three datasets while enabling 2.77 times smaller cache size. Reduced memory footprint with AttentionPack enables larger

Model	Method	Cache Memory (avg. per instance) Size (MB)	Reduction	Batch Inf. Average Change	Video QA Datasets MSVD-QA (acc.)	MSRVTT-QA (acc.)
VideoLLaVA-7B	Full KV Caching	1114.6	1×	–	69.33	55.60
	FastV [5] ($k = 5, e = 50\%$)	1114.6	1×	+23%	**69.60**	**55.52**
	Scissorhands [21] ($e = 50\%$)	557.3	2×	+32%	65.90	54.33
	H2O [32] ($e = 50\%$)	557.3	2×	+32%	66.54	54.80
	AttentionPack ($R_{kv} = R_{vv} = 128$)	**137.5**	**8.11×**	**+60%**	69.21	55.47

Table 2. Video QA results on MSVD-QA and MSRVTT-QA with VideoLLaVA-7B. We report evaluation metrics along with cache memory and throughput statistics for full KV caching, H2O eviction, FastV to compare with our approach AttentionPack.

Key Cache R_{kv}	Key Cache Red.	Value Cache R_{vv}	Value Cache Red.	Total Cache Size (GB)	Total Cache Red.	Dataset AOKVQA	Dataset OCRVQA
	–	–	–	10.50	–	76.64	51.05
128	3.19×	256	1.83×	4.51	2.32×	76.77	52.50
		128	3.19×	3.28	3.19×	76.80	52.51
64	5.09×	128	3.19×	2.68	3.92×	76.75	52.47
		64	5.09×	2.06	5.09×	76.88	52.44
32	7.24×	64	5.09×	1.75	5.98×	76.46	52.00
		32	7.24×	1.45	7.24×	75.91	51.69
16	9.18×	32	7.24×	1.29	8.09×	74.12	50.06
		16	9.18×	1.14	9.18×	72.13	48.44
$16 \rightarrow 128$	4.78×	$32 \rightarrow 256$	2.95×	2.87	3.65×	74.33	50.85
		$16 \rightarrow 128$	4.78×	2.19	4.78×	72.51	48.80
64 (90%)	5.31×	64 (90%)	5.16×	2.00	5.23×	76.50	52.18

Table 3. Performance and cache size for various compression rank values using LLaVA1.5-7B for batch size of 32.

batch sizes, increasing the throughput in batch inference up to 61% for the rank of 64. We have similar observation for video QA as reported in Table 2. AttentionPack performs 0.39% and 0.05% worse compared to FastV while having 8.11 times smaller cache. In video QA, the size of visual token cache increases by the number of frames. Supported by the fact that most frames carry similar information, we observe that higher rates of compression is achievable without resulting in performance decrease.

4.3. Analysis of Rank and Decompression

Impact of Compression Rate: We analyze the impact of compression rank values for key and value caches in Table 3 with LLaVA1.5-7B. We observe that for rank values below 64, the performance starts to drop for both datasets while increasing the rank from 64 to 128 brings negligible performance improvements. At $R_{kv} = R_{vv} = 64$, we observe 0.24% performance drop in A-OKVQA and 1.39% performance increase in OCR-VQA while having around five times smaller cache, showing that based on the dataset, compression can actually filter out the irrelevant information within the visual input and enhance performance.

In addition, we reported the results for two other rank selection schemes, at the last two rows of Table 3. First, we

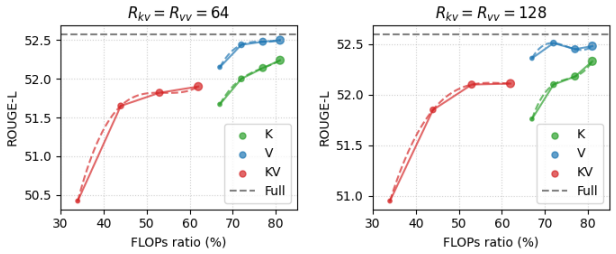


Figure 4. Impact of attention-aware decompression. Each line represents the results when AttentionPack is applied for key (k), value (v) caches or both (kv). Every line has four dots with the size of each representing the ratio of visual tokens ($r_1 \in \{0.125, 0.25, 0.375, 0.5\}$) decompressed with the full rank.

set the rank with a linearly increasing value such that the key cache in the first layer is compressed with the rank of 16 and last with 128. We observe that the performance is 0.38% and 0.36% better across two datasets compared to directly setting the rank of each layer to 16, while requiring almost twice the size of memory. We see that the ranks selected for earlier layers tend to dominate how much the model output quality will be preserved. Second, we set the rank of each layer based on the explained variance ratio such that the preserved ratio will be above 90% if possible without exceeding a maximum rank value to 64. Based on the data, this approach may enable setting a lower rank value to certain layers with key and value vectors having intrinsic lower ranks. However, we observe around 0.3% performance drop.

Impact of Attention-Aware Decompression: We analyze the impact of attention-aware decompression in Table 4 with LLaVA1.5-7B on OCR-VQA. Here, x-axis show the ratio of decompression FLOPs compared to the FLOPs without attention-aware decompression. Every line has four dots with the size of each representing the ratio of visual tokens (r_1) decompressed with the full compression rank $R_{kv} = R_{vv} = 64$ (left), $R_{kv} = R_{vv} = 128$ (right), while the rest ($r_2 = 1 - r_1$) being decompressed with $R_{kv}/4$ and $R_{vv}/4$. We experiment with $r_1 \in \{0.125, 0.25, 0.375, 0.5\}$. For instance, for the compression rank of 64, we observe that decompressing the 25% of value cache with the rank of

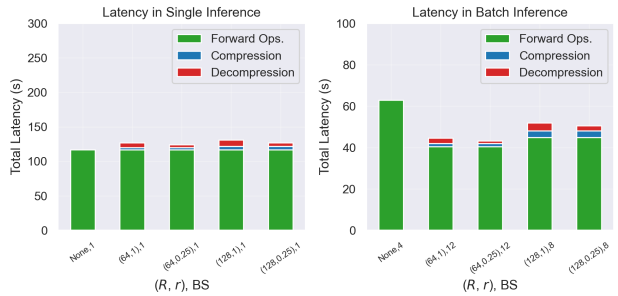


Figure 5. Total decode latency for 100 queries with LLaVA1.5-7B.

Method	Avg. Cache Size (MB)	Avg. Thr. Change	Dataset A-OKVQA	Dataset OCR-VQA
Full KV - fp16	328.2	–	76.64	51.05
KVQuant - 4bit [13]	82.1	+49%	75.90	50.67
AttentionPack	64.5	+54%	**76.88**	**52.44**
AttentionPack (E)	62.1	+70%	**76.88**	51.63
AttentionPack - 4bit	16.1	+97%	75.27	50.18
AttentionPack - 4bit (E)	**15.5**	**+115%**	75.27	49.11

Table 4. Collected results (accuracy on A-OKVQA rouge-l on OCR-VQA) for LLaVA1.5-7B with measured average cache size per instance (MB) and avg. throughput change in batch inference.

64 achieves almost the same performance with full decompression while having almost 30% less FLOPs computed for decompression. However, the key vectors are more sensitive to the information loss after partial decompression as we observe higher performance drop in those scenarios, potentially due to the impact on attention weight calculations, which also affects the tokens that are fully decompressed.

4.4. Computational Cost Analysis

In Figure 5, we present the total decoding latency for 100 queries at OCR-VQA using LLaVA1.5-7B. We conduct measurements across different compression ranks and decompression ratios to analyze how latency varies with these configurations. The top-row plots illustrate latency in the single-inference scenario, while the bottom-row plots depict latency when the batch size is maximized to fully utilize the available GPU memory. By reducing the cache size per instance by approximately 80% through compression, we achieve around 4 times larger batch sizes, leading to a potential reduction in total latency of up to 54%. Latency measurements in this subsection are performed with 4-bit model weight loading and half-precision data. For these measurements, we used RTX3060, as it enabled granular monitoring of each operation (forward ops., compression, decompression) with local access. We also provide a similar analysis at various context lengths in Appendix 8.1.

4.5. Compatibility with Other Techniques

Integration with Eviction and Quantization: To analyze the potential of our technique, we perform experiments combining AttentionPack with eviction and also quantization. It is important to explore the complementary behavior of technique, especially for environments with strict memory restriction that may need aggressive cache size reduction. To this end, we first replace attention-aware decompression with eviction such that the tokens determined to be decompressed with lower rank based on Eq. 1 are evicted. We also analyze the performance in combination with 4-bit quantization following KVQuant [13]. As reported in Table 4, we observe that AttentionPack can achieve further gains when combined with quantization and eviction. We see around 0.5% performance drop compared to KVQuant when we combine quantization with our approach while having $5\times$ smaller cache and $2\times$ faster inference. Eviction affects the performance negatively in OCR-VQA as the visual tokens needed for accurate answer may change at each step.

Method	Cache Red.	MMMU Acc.
Full KV Caching	-	69.6%
FastV [5] ($k = 2, e = 50\%$)	$1\times$	68.1%
H2O [32] ($e = 50\%$)	$2\times$	66.9%
AttentionPack ($R_{kv} = R_{vv} = 64$)	$3.17\times$	67.9%

Table 5. Image QA results (average cache size reduction per query vs accuracy) on MMMU with Qwen3VL-8B-instruct.

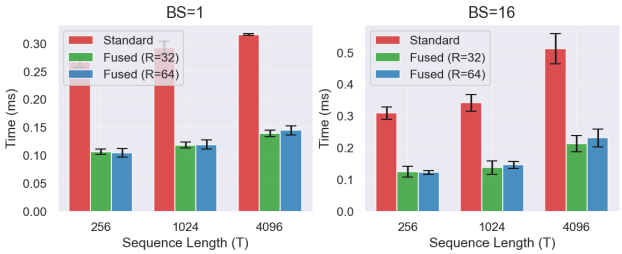


Figure 6. Decoding latency for 32 tokens with various batch sizes and sequence lengths using standard attention and with fused kernel implementation of AttentionPack. We observe up to 2.4x faster inference in single and batch query settings.

Integration with Grouped Query Attention: Groped query attention (GQA) is a variation of the attention mechanism used in transformer models, designed to improve efficiency and scalability. GQA reduces the computational cost by having fewer key-value groups than query heads. To analyze the compatibility of our approach with GQA, we perform experiments with the recent video-language model Qwen3VL-8B-instruct, which utilizes this technique. In Table 5, we also report the results obtained on MMMU with competitive performance at 3.17x smaller cache.

Integration with Lower-level Optimizations: To achieve further latency reductions, we also demonstrate that integration with low-level optimizations such as FlashAttention [8] is also possible. To this end, we provide a fused kernel implementation, which merges the decompression operation

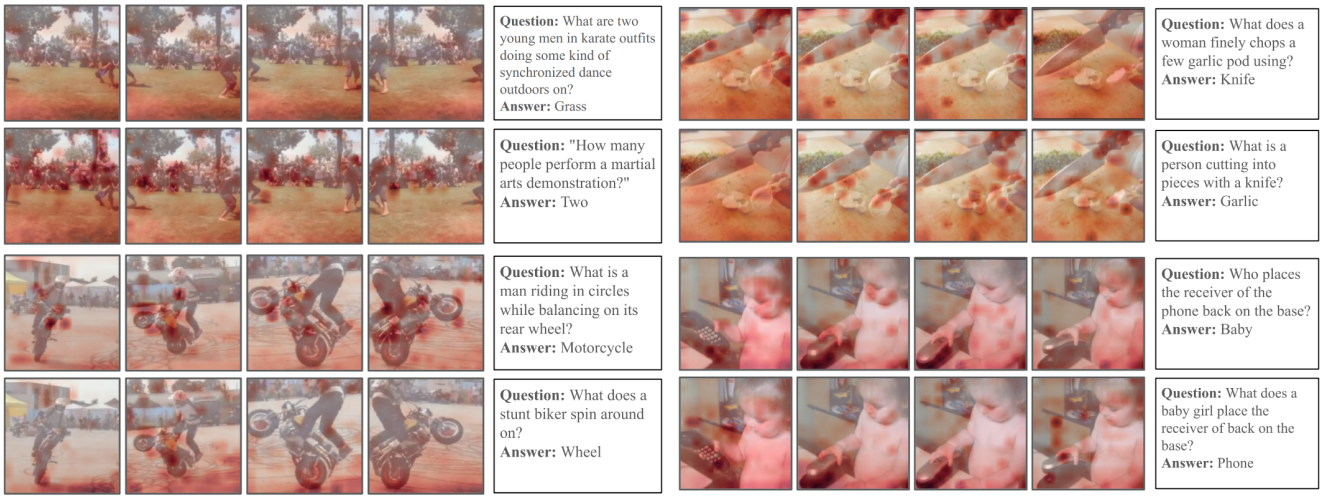


Figure 7. Visualization of scores (Eq. 1) at the first decoding step after processing the question on various examples from MSVD-QA, showing that the visual tokens closely related to the prompt have higher importance scores and will be decompressed with higher ranks.



Figure 8. Visualization of tracked attention-score statistics (Eq. 1) on various image QA pairs from OCR-VQA and A-OKVQA.

directly with the attention score computation (Algorithm 1, lines 7-12). The principle is similar to FlashAttention [8], which optimizes I/O by reducing data transfers between HBM and SRAM. We apply this idea by incorporating our decompression step into FlashAttention-v1. This approach successfully reduces latency while still permitting the materialization of attention scores for our importance estimation. Fused kernel optimizes memory access by for each query, tiling the key and value cache blocks along the sequential axis such that the decompression and multiplication happens on-the-fly in SRAM together just before the attention score calculation with online softmax. Empirical results on LLaVA1.5-7B (Fig 6) confirm the benefits, where the fused kernel almost halves the decoding latency for 32 tokens in both single and batch inference settings when compared to our standard implementation.

4.6. Qualitative Analysis

To understand how the attention score statistics utilized in attention-aware decompression, we illustrate several image-question pairs from MSVD-QA in Figure 7 and OCR-VQA, A-OKVQA in Figure 8. We observe that the tracked importance scores highlights various regions of the visual input depending on the question and not every visual token is equally used at each decoding step. The model tends to put less attention on backgrounds and more on the objects related to the desired answer. These examples show the validity of the motivation behind our attention-aware decompression technique, which reduces decompression overhead without hurting output quality.

5. Conclusion

We have proposed AttentionPack, an attention-aware memory-efficiency optimization framework for decoding with large vision-language models. Our approach introduces two key technical contributions: (1) a multi-head compression technique leveraging singular value decomposition to reduce the size of stored key and value vectors, and (2) an attention-aware partial decompression technique, minimizing latency overhead and addressing memory constraints in long-context inference. Thanks to reduced memory usage, AttentionPack enables larger batch sizes and extended context lengths while preserving the model output quality and performance. Comprehensive evaluations on five image and video QA datasets demonstrate that our approach can reduce the key-value cache size by up to 8 times and improve batch inference throughput by up to 50%.

Acknowledgement. This research is partially sponsored by NSF CISE grants 2302720 and 2312758, an IBM faculty award, a grant from the CISCO Edge AI program, and the research cyberinfrastructure resources and services provided by the Partnership for an Advanced Computing Environment (PACE) at the Georgia Institute of Technology. Fatih Ilhan and Ling Liu are the primary contact authors for this work.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. 2

[2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023. 2, 5, 11

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *CoRR*, abs/2004.05150, 2020. 2

[4] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, and Kai-Chiang Wu. Palu: Compressing kv-cache with low-rank projection, 2024. 2

[5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models, 2024. 2, 5, 6, 7, 11

[6] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023. 11

[7] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. *CoRR*, abs/2009.14794, 2020. 2

[8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: fast and memory-efficient exact attention with io-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2024. Curran Associates Inc. 2, 7, 8

[9] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, and Aiesha Letman et al. The llama 3 herd of models, 2024. 12

[10] Bai et al. LongBench: A bilingual, multitask benchmark for long context understanding, ACL 2024. 12

[11] Shuai Bai et al. Qwen3-vl technical report, 2025. 5

[12] N. Halko, P. G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2): 217–288, 2011. 11

[13] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024. 1, 2, 7

[14] Fatih Ilhan, Gong Su, Selim Furkan Tekin, Tiansheng Huang, Sihao Hu, and Ling Liu. Resource-efficient transformer pruning for finetuning of large models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16206–16215, 2024. 2

[15] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm, 2024. 1, 2

[16] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020. 2

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. 2

[18] Bin Lin, Bin Zhu, Yang Ye, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 1, 2

[19] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models. In *Advances in Neural Information Processing Systems*, pages 139997–140031. Curran Associates, Inc., 2024. 2, 5, 11

[20] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023. 1, 2, 3, 5, 11

[21] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. In *Advances in Neural Information Processing Systems*, pages 52342–52364. Curran Associates, Inc., 2023. 1, 2, 5, 6, 11

[22] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-pruner: On the structural pruning of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 2

[23] Anand Mishra, Shashank Shekhar, Ajeet Kumar Singh, and Anirban Chakraborty. Ocr-vqa: Visual question answering by reading text in images. In *ICDAR*, 2019. 3, 5, 11

[24] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021. 11

[25] Dustin Schwenk, Apoorv Khandelwal, Christopher Clark, Kenneth Marino, and Roozbeh Mottaghi. A-okvqa: A bench-

mark fornbsp;visual question answering using world knowledge. In *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part VIII*, page 146–162, Berlin, Heidelberg, 2022. Springer-Verlag. 5, 11

[26] Gemini Team. Gemini: A family of highly capable multimodal models, 2024. 1

[27] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. Look-m: Look-once optimization in kv cache for efficient multimodal long-context inference. In *Conference on Empirical Methods in Natural Language Processing*, 2024. 2

[28] Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *ACM Multimedia*, 2017. 5, 11

[29] Hongxu Yin, Arash Vahdat, Jose Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-ViT: Adaptive tokens for efficient vision transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[30] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. Asvd: Activation-aware singular value decomposition for compressing large language models, 2024. 2

[31] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of CVPR*, 2024. 5, 11

[32] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, pages 34661–34710. Curran Associates, Inc., 2023. 1, 2, 5, 6, 7, 11, 12

[33] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. 12

[34] Bin Zhu, Bin Lin, Munan Ning, Yang Yan, Jiaxi Cui, HongFa Wang, Yatian Pang, Wenhao Jiang, Junwu Zhang, Zongwei Li, et al. Languagebind: Extending video-language pretraining to n-modality by language-based semantic alignment. *arXiv preprint arXiv:2310.01852*, 2023. 11